Lowlights

Five things that went wrong while building LionsList with an agent — and how I caught them.

VINAYAK RAJU · LIONSLIST / COLUMBIA MARKET · SEPTEMBER 2026

The agent almost never failed loudly. It failed *plausibly* — confident prose, clean code, a number wrong only in the sentence around it. None of the items below announced itself as an error. Each came out of checking something I had no reason to doubt.

1. I mistook a long spec for a clear one

NOTICED: early builds kept drifting from what the team had agreed. The agent wasn't ignoring the spec — it was sampling different parts of it each time.

WENT WRONG: our proposal ran to 51 pages. That is not more context, it is more surface area. The load-bearing parts — data model, the four allowed email domains, the feed query — sat buried among material that constrained nothing.

DID: cut it to the decisions that actually bind the build, and moved those constraints into the prompt itself instead of trusting the agent to go find them.

2. It picked the architecture for me, silently

NOTICED: reading the generated scope, the API was TypeScript and Drizzle. I had never asked for that — but I had never asked for Python either.

WENT WRONG: I left the stack as an open question, so the agent closed it with its own default and moved on. By the time I looked, Codex had already scaffolded a working Next.js MVP down the wrong road.

DID: pushed back directly — "*why does it reject python, can't we use a python backend?*" — had the spec rewritten around a Python API and Python analytics, and rebuilt on FastAPI. Any decision I don't make, the agent makes for me — and never flags that it did.

3. Every number on the dashboard was right; the sentence above them was wrong

NOTICED: I didn't trust the headline tiles, so I re-ran the same aggregation straight against SQLite. I got 29 listed and 8 sold; the page claimed 53 and 21.

WENT WRONG: pandas `resample("W")` labels each bucket by the week's *end* date. The figures were correct — for 24–30 August. The page called it "the week beginning 2026-08-30". No test fails and no error is raised: the analysis is sound and the label lies.

DID: traced it to the resample rule in the insights router and fixed the label rather than the buckets, since the same off-by-one was on the chart axis and tooltips too.

4. Search fails on the one word the product teaches you

NOTICED: I typed "textbook" into my own feed and got nothing back — while a **TEXTBOOKS** filter chip sat on the same screen.

WENT WRONG: the query matches title and description, never category. 273 listings sit in that category and not one contains the word. The event log settles it: "textbook" is the most common zero-result search (82 times), and 33% of all searches return nothing.

DID: diagnosed it, and deliberately did *not* patch it yet. That 33% is reported in our analytics as a measured finding, so fixing search moves a number we present as evidence. It needs a before-and-after, not a silent fix.

5. The agent's account of my own work was wrong

NOTICED: compiling my transcripts for submission, I read the rendered output instead of trusting it. A "prompt" attributed to me opened "*Approach this as the design lead at a small studio…*". I never wrote that sentence.

WENT WRONG: instructions the harness injects arrive in the transcript as user turns. The export counted them as mine — five of them.

DID: filtered on the flag that marks injected content and rebuilt; my prompt count fell from 55 to 50. On a graded artifact, that gap is the difference between an accurate record and a flattering one.

What I changed in how I work

Model selection is a budget decision, not a quality one. Multi-agent fan-out and the heaviest models are real capability, but a CRUD marketplace with a settled spec does not need them. Opus did this job; reaching further mostly bought tokens.

Sequential sessions over parallel agents. Separate focused sessions kept context small enough to stay accurate and made spend legible; fanning out agents burned budget on work I then had to re-read anyway.

Walk the whole user cycle by hand. Items 3–5 were found by using the thing, not reading its code — and a sixth (sign-in "bad credentials" that was an *empty* password field, not a wrong one) came from reading the mailer instead of believing the error. The agent's first diagnosis is a hypothesis.

> *The through-line: agents are strong once the plan and architecture are genuinely settled, and quietly inventive before that. My job was not writing the code. It was noticing the places where something plausible had been substituted for something true.*